INSTITUTE OF ENGINEERING & MANAGEMENT
Good Education, Good Jobs

PCCCS495 : Advanced Programming(OOP) Lab Project

Semester: 4

HOTEL BOOKING SYSTEM

Name: Dibyajyoti Saha

Roll No.: 47

Enrollment No.: 12024052017015

Course: Bachelor of Technology

Stream: CSE (IOTCSBT)

Instructors:

Dr. Swagatam Basu, Dr. Susovan Jana, Dr. Soumadip Biswas

SPRING 2026

Problem Statement

Managing hotel room bookings manually can be inefficient, error-prone, and time-consuming, especially for small hotels or guest houses that handle multiple reservations daily. Maintaining records of room availability, bookings, and cancellations without a proper system often leads to issues such as double booking, incorrect room allocation, and difficulty in tracking current room status.

The objective of this project is to design and implement a console-based Hotel Booking System in Java that simplifies and automates the process of managing hotel room reservations. The system should allow users to view available rooms, check detailed information such as room number, type, price, and booking status, book rooms for customers, and cancel existing bookings. It must also handle common errors, such as attempting to book an already reserved room or entering an invalid room number.

The system will be developed using Object-Oriented Programming principles to ensure modularity, reusability, and maintainability. An abstract base class will define common properties and behaviours of rooms, while specific room types such as Single Room and Double Room will be implemented using inheritance. Polymorphism will be used to handle different room types uniformly, and encapsulation will protect internal data through controlled access methods. Exception handling will be incorporated to manage runtime errors effectively.

The final application will be menu-driven and user-friendly, enabling efficient interaction and smooth operation. The system aims to provide a simple yet structured solution for hotel staff to manage room bookings effectively while demonstrating core programming concepts.

System Design

UML Class Diagram

The diagram below shows all classes, their attributes, methods, and relationships. Visibility prefixes follow UML convention: + public, - private, # protected.

▼ **extends** ▼

«abstract»
Room
- roomNumber : int
- isBooked :boolean
+ Room(roomNumber : int)
+ getRoomNumber() : int
+ isBooked() :boolean
+ bookRoom() :void «throws BookingException»
+ cancelBooking() :void «throws BookingException»
+ displayDetails() :void «abstract»

▼ ▼

«concrete»
SingleRoom
(no additional fields)
+ SingleRoom(roomNumber : int)
+ displayDetails() : void

«concrete»
DoubleRoom
(no additional fields)
+ DoubleRoom(roomNumber : int)
+ displayDetails() : void

Hotel
- rooms : List<Room>
+ Hotel()
+ addRoom(room : Room) : void
+ getRooms() : List<Room>
+ viewRooms() : void
+ findRoom(roomNumber : int) : Room
+ bookRoom(roomNumber : int) : void
+ cancelBooking(roomNumber : int) : void

«exception»
BookingException
+ BookingException(message : String)

«utility»
DataManager
+ FILE_NAME : String = "rooms.txt"
+ saveRooms(rooms : List<Room>) :void «static»
+ loadRooms() : List<Room> «static»

«entry point»
Main
+ main(args :String[]) :void «static»

Description of Major Classes

Room (Abstract Class)
Defines the common properties of all rooms such as room number, price, and booking status. It also provides basic methods for booking and cancelling, and declares an abstract method displayDetails() to be implemented by subclasses.

SingleRoom and DoubleRoom
These classes extend the Room class and represent specific types of rooms. They override the displayDetails() method to show customized information for each room type.

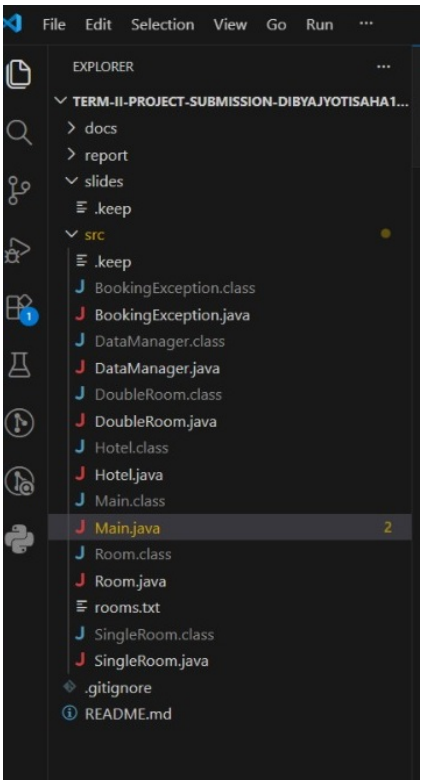
Hotel
Acts as the central management class. It stores a collection of Room objects and provides methods to add rooms, view all rooms, find a specific room, book a room, and cancel bookings.

BookingException
A custom exception class used to handle errors such as attempting to book an already booked room or cancelling a non-booked room.

DataManager
Handles file operations for saving and loading room data. It ensures that booking information persists between program executions.

Main
Contains the main method and provides a menu-driven interface for user interaction. It connects all components and controls the overall program flow.

Package Structure



Interaction Flow

1. The program starts from the Main class.
2. Existing room data is loaded using DataManager (if available).
3. The system displays a menu to the user.
4. The user selects an option:
    - View rooms → Hotel.viewRooms()
    - Book room → Hotel.bookRoom()
    - Cancel booking → Hotel.cancelBooking()
5. The Hotel class interacts with Room objects to perform operations.
6. Exceptions are handled using BookingException to ensure safe execution.
7. On exit, room data is saved using DataManager.

This structured flow ensures smooth interaction between different components and efficient management of hotel bookings.

OOP Concepts Demonstrated

1. Abstraction

Where it is used
In the Room abstract class.

Why it is used
To define a common structure for all room types while hiding implementation details.

Code Snippet

```
public abstract class Room {

    private int roomNumber;

    private double price;

    private booleanisBooked;

    public Room(int roomNumber, double price) {

this.roomNumber = roomNumber;

this.price = price;

this.isBooked = false;

    }

    public abstract void displayDetails();

}
```

Explanation
The Room class provides common properties and declares an abstract method displayDetails(). This ensures all subclasses must implement their own version of this method while sharing common attributes.

Inheritance

Where it is used
SingleRoom and DoubleRoom classes extend the Room class.

Why it is used
To reuse common code and avoid duplication.

Code Snippet

```
public class SingleRoom extends Room {

    public SingleRoom(int roomNumber) {

super(roomNumber, 1000);

    }

    @Override
```

```
    public void displayDetails() {
System.out.println("Single Room: " + getRoomNumber());
    }
}
```

Explanation
The SingleRoom class inherits properties and methods from Room and only adds specific behavior, reducing redundancy and improving maintainability.

Polymorphism

Where it is used
In the Hotel class using List<Room>.

Why it is used
To treat different room types uniformly through a common base class.

Code Snippet

```
List<Room> rooms = new ArrayList<>();
rooms.add(new SingleRoom(101));
rooms.add(new DoubleRoom(201));
for (Room room : rooms) {
room.displayDetails();
}
```

Explanation
Although the list contains different types of objects, they are treated as Room. The correct displayDetails() method is called at runtime based on the actual object type.

Encapsulation

Where it is used
In the Room class with private variables and public methods.

Why it is used
To protect data and control access to class attributes.

Code Snippet

```
private int roomNumber;
public int getRoomNumber() {
    return roomNumber;
}
public void bookRoom() {
this.isBooked = true;
}
```

Explanation
The data is hidden using private variables and accessed through methods. This ensures controlled modification and improves data security.

Exception Handling

Where it is used
In booking and cancellation operations using BookingException.

Why it is used
To handle errors gracefully and prevent program crashes.

Code Snippet

```
public void bookRoom() throws BookingException {
   if (isBooked) {
      throw new BookingException("Room already booked!");
   }
isBooked = true;
}
```

Explanation
If a room is already booked, an exception is thrown. This prevents invalid operations and ensures the system behaves reliably under error conditions.

Implementation Highlights

1. Centralized Room Management using List

Code Snippet

```
List<Room> rooms = new ArrayList<>();
public void addRoom(Room room) {
rooms.add(room);
}
```

Justification
Using a List<Room> allows the system to store different types of rooms (SingleRoom and DoubleRoom) in a single collection. This design supports scalability and enables easy addition of new room types without modifying existing logic.

Booking Logic with Validation

Code Snippet

```
public void bookRoom(int roomNumber) throws BookingException {
   Room room = findRoom(roomNumber);
   if (room == null) {
      throw new BookingException("Invalid room number!");
```

```
    }
room.bookRoom();
}
```

Justification
This method ensures that only valid rooms can be booked. It first checks whether the room exists and then delegates the booking operation. This separation of concerns improves code clarity and reliability.

Runtime Polymorphism in Display

Code Snippet

```
public void viewRooms() {
    for (Room room : rooms) {
room.displayDetails();
    }
}
```

Justification
This demonstrates runtime polymorphism, where different room types are treated uniformly as Room objects. The correct method implementation is executed dynamically, making the system flexible and extensible.

File-Based Data Persistence

Code Snippet

```
BufferedWriter writer = new BufferedWriter(new FileWriter("rooms.txt"));
for (Room r : rooms) {
writer.write(r.getRoomNumber() + "," + r.getPrice() + "," + r.isBooked());
writer.newLine();
}
writer.close();
```

Justification
This implementation allows room data to be saved to a file, ensuring that booking information is preserved between program runs. It enhances the practicality of the system beyond a temporary in-memory solution.

Menu-Driven User Interaction

Code Snippet

```
while (true) {
System.out.println("1. View Rooms");
System.out.println("2. Book Room");
```

```
System.out.println("3. Cancel Booking");
System.out.println("4. Exit");
    int choice = sc.nextInt();
}
```

Justification
A loop-based menu system provides continuous interaction with the user. It ensures usability and allows multiple operations without restarting the program, making the application user-friendly.

Testing and Error Handling

Test Cases Considered

- Viewing all rooms when the system starts
- Booking an available room successfully
- Cancelling an existing booking
- Booking multiple rooms sequentially
- Verifying room status updates after booking and cancellation
- Loading previously saved room data (if persistence is enabled)

Edge Cases Handled

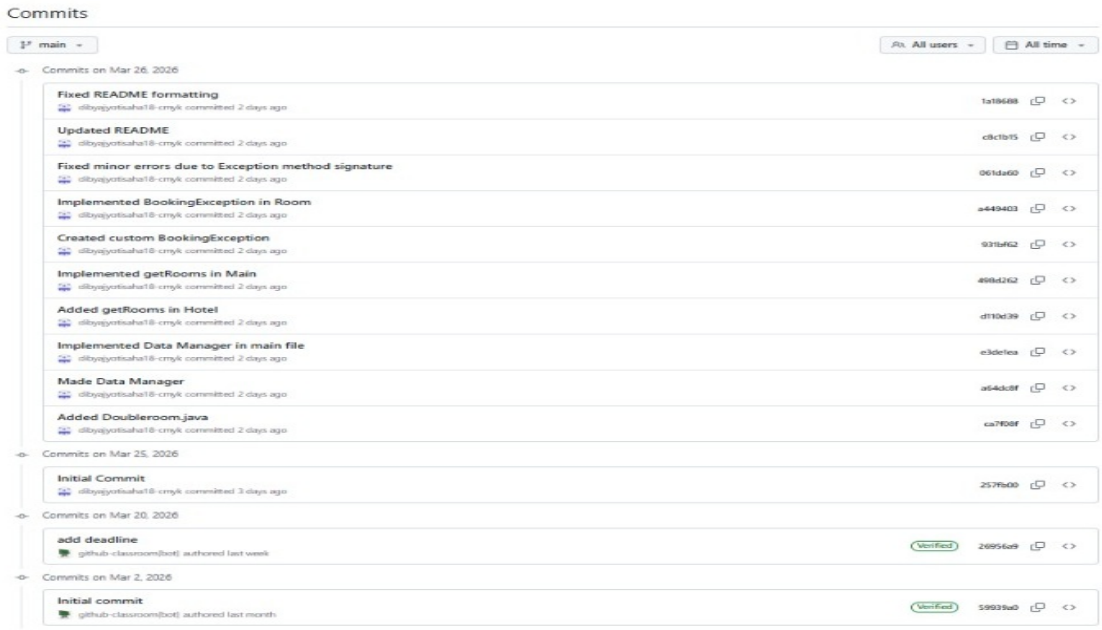
- Attempting to book a room that is already booked
- Attempting to cancel a room that is not booked
- Entering an invalid room number (room does not exist)
- Handling empty or missing data file during initial load
- Repeated user inputs without restarting the application

Failure Scenarios

- Invalid room number results in a BookingException with an appropriate message
- Booking an already reserved room triggers an exception and prevents duplication
- Cancelling a non-booked room results in an error message
- File read/write errors are handled using try-catch blocks to prevent application crash
- Incorrect user input does not terminate the program and is handled gracefully

Git Workflow

Commit History



Conclusion and Future Scope

Conclusion

The Hotel Booking System successfully demonstrates the use of Object-Oriented Programming principles to build a modular and functional application. The system allows users to efficiently manage room bookings, view room details, and handle cancellations through a simple console-based interface. Key OOP concepts such as abstraction, inheritance, polymorphism, encapsulation, and exception handling are effectively implemented, making the system structured and easy to maintain.

Future Scope

The current system can be further enhanced in several ways:

- **Graphical User Interface (GUI)**
  Replace the console-based interface with a graphical interface for better user experience.
- **Database Integration**
  Use a database such as MySQL instead of file storage for more efficient and scalable data management.
- **Customer Management**
  Store customer details such as name, contact information, and booking history.
- **Online Booking System**
  Extend the application to support web-based or mobile-based booking.
- **Advanced Features**
  Add functionalities like room availability filters, pricing variations, and booking reports.

These enhancements can transform the system into a more comprehensive and real-world hotel management solution.

Appendices

Copy of the initial project proposal

https://drive.google.com/file/d/1XQP4wbWA2K8LSJHvhNeAFdoscVF6kLNo/view?usp=drivesdk

Copy of the reviewed mail received

Project: Hotel Booking System
Decision: APPROVED_MINOR
Score: 88

Strengths:
The proposal clearly defines the problem, target a
and core features for a hotel booking system. It ex
lists all key OOP concepts (Abstraction, Inheritanc
Polymorphism, Encapsulation, Exception Handlin
outlines a logical architecture that demonstrates t
principles, particularly with the abstract 'Room' cl
concrete subclasses. The inclusion of custom exc
handling for specific error conditions is a strong p
modular, menu-driven approach is appropriate fo
proposed console application and ensures a clea
separation of concerns.

Improvements:
The description of polymorphism in the Architectu
could be refined for greater clarity. While 'adding c
achieved via inheritance, polymorphism is more a
treating objects of different types uniformly throug
common interface or base type (e.g., iterating thro
of 'Room' objects that are actually 'SingleRoom' o